Projet Constraint Programming (PyCSP3)

Maryem Minouari

April 10, 2026

Contents

1 Frequency Allocation **2**
1.1 Description du problème 2
1.2 Modélisation PyCSP3 2
1.3 Difficultés rencontrées 2
1.4 Expérimentations 3

2 Meeting Scheduling **3**
2.1 Description du problème 3
2.2 Modélisation PyCSP3 3
2.3 Problèmes rencontrés 4
2.4 Expérimentations 4

1 Frequency Allocation

1.1 Description du problème

Le problème de Frequency Allocation consiste à attribuer une fréquence entière dans un domaine $1 \dots F$ à chaque émetteur réparti dans des cellules, en respectant deux types de contraintes : deux émetteurs d'une même cellule doivent avoir des fréquences espacées d'au moins 16, et pour deux cellules différentes, une matrice symétrique impose une distance minimale entre les fréquences de leurs émetteurs respectifs. L'objectif est de minimiser la fréquence maximale utilisée, ce qui revient à minimiser le spectre fréquentiel occupé.

L'objectif est de minimiser le nombre de fréquences distinctes utilisées (ou, de manière équivalente, la fréquence maximale utilisée).

1.2 Modélisation PyCSP3

Soit n le nombre total d'émetteurs. On note $c(i)$ la cellule de l'émetteur i et $t(i)$ son numéro dans cette cellule. La fréquence de l'émetteur i est notée x_i.

- **Variables :** On définit un tableau de n variables entières, une par émetteur :

  ```
  freq = VarArray(size=n, dom=range(1, nbFreqs + 1))
  ```

  où `freq[i]` représente la fréquence assignée à l'émetteur i, et le domaine de chaque variable est l'ensemble des fréquences disponibles $\{1, 2, \dots, F\}$, avec F le nombre total de fréquences autorisées.

- **Contraintes intra-cellule :** Pour chaque cellule c et chaque paire d'émetteurs (t_1, t_2) de cette cellule :

  $$|x_{i(c,t_1)} - x_{i(c,t_2)}| \geq 16$$

- **Contraintes inter-cellules :** Pour deux cellules c_1, c_2 ($c_1 < c_2$) et pour tous émetteurs respectifs :

  $$|x_{i(c_1,t_1)} - x_{i(c_2,t_2)}| \geq \text{distance}[c_1][c_2]$$

- **Rupture de symétrie :** On fixe la fréquence du premier émetteur à 1 pour casser les permutations de fréquences :

  ```
  satisfy(freq[0] == 1)
  ```

- **Objectif :** Minimiser la fréquence maximale utilisée, ce qui revient à minimiser le nombre de fréquences distinctes (car les fréquences sont consécutives à partir de 1)

1.3 Difficultés rencontrées

- **Tentative d'intégration de Choco :** l'objectif était de comparer ACE et Choco sur les mêmes instances avec un timeout identique. Cependant, Choco retournait systématiquement UNSAT en moins de 2 secondes, ce qui est incohérent puisqu'ACE trouvait des solutions valides sur les mêmes instances. Les causes identifiées sont : une syntaxe d'options incompatible entre les versions de pycsp3, l'absence de fichier de log parsable pour récupérer une solution intermédiaire en cas de timeout, et des différences internes dans l'interprétation du modèle. Ces problèmes ont conduit à concentrer l'expérimentation uniquement sur ACE.

1.4 Expérimentations

Table 1: Performances d'ACE pour différents timeouts sur l'instance d'allocation de fréquences

Timeout (s)	**Temps réel (s)**	**Fails**	**Ngds**	**Objectif**
800	636.92	4937050	6105	149
1200	881.75	5590924	6347	149
3600	2801.33	18356228	13940	149

Commentaire : Quelle que soit la durée d'exécution (de 800 s à 3600 s), le solveur n'a trouvé aucune solution améliorant la fréquence maximale (toujours 149). Le nombre de nœuds (`Ngds`) et d'échecs (`Fails`) augmente avec le temps, mais l'objectif reste bloqué, suggérant que la solution de valeur 149 est très probablement optimale pour cette instance.

2 Meeting Scheduling

2.1 Description du problème

Le problème de Meeting Scheduling consiste à assigner un créneau horaire à chaque réunion parmi un ensemble de m réunions, en tenant compte des contraintes de disponibilité des agents et des distances de déplacement entre les réunions. Chaque agent doit assister à un sous-ensemble fixe de réunions, et deux réunions consécutives auxquelles participe un même agent doivent être séparées d'un intervalle de temps suffisant pour permettre le déplacement entre leurs lieux respectifs. Ce délai minimal est donné par une matrice de distances symétrique entre les réunions. L'objectif est d'assigner à chaque réunion un créneau dans un domaine discret $\{1, \ldots, D\}$ tout en minimisant les conflits ou en satisfaisant toutes les contraintes simultanement.

2.2 Modélisation PyCSP3

Variables : on définit un tableau de m variables entières, une par réunion, où `x[i]` représente le créneau horaire assigné à la réunion i dans le domaine $\{0, 1, \ldots, D-1\}$:

```
x = VarArray(size=nMeetings, dom=range(domainSize))
```

Contraintes : pour chaque agent et chaque paire de réunions (i, j) auxquelles il participe, les deux réunions doivent être séparées d'un intervalle suffisant :

$$|x_i - x_j| \geq d_{ij} + 1$$

Le $+1$ vient du fait que les créneaux sont des unités de temps discrètes et indivisibles. Si la réunion i est planifiée au créneau a et la réunion j au créneau $b > a$, alors $b - a$ est le nombre de créneaux qui les séparent. La réunion i occupe entièrement le créneau a, donc le déplacement ne peut commencer qu'au créneau $a + 1$ au plus tôt. Il faut ainsi que $b - a \geq d_{ij} + 1$, ce qui explique le $+1$: un créneau est consommé par la réunion elle-même avant que le déplacement puisse commencer. En PyCSP3, cette contrainte est exprimée sous forme disjonctive car on ne connaît pas à l'avance laquelle des deux réunions est planifiée en premier :

```
satisfy(
    [(x[i] - x[j] >= dist[i][j] + 1) |
     (x[j] - x[i] >= dist[i][j] + 1)
     for meetings in agents_meetings
     for i in meetings for j in meetings if i < j]
)
```

Objectif : on minimise le makespan, c'est-à-dire le créneau maximum utilisé parmi toutes les réunions. Minimiser ce créneau revient à serrer au maximum les réunions dans le temps tout en respectant les contraintes de déplacement, ce qui permet d'obtenir un planning le plus compact possible :

```
makespan = Var(dom=range(domainSize))
satisfy(makespan == Maximum(x))
minimize(makespan)
```

2.3 Problèmes rencontrés

Instances non faisables : certaines instances n'admettent aucune solution dans le domaine $\{0, \ldots, D-1\}$ fourni, car les contraintes de déplacement cumulées entre les réunions d'un même agent dépassent la taille du domaine disponible.

Astérisques dans les solutions ACE : lors de la récupération de la solution, certaines variables retournaient la valeur `*` au lieu d'un entier, ce qui rendait la solution inexploitable. Ce comportement survenait lorsque ACE atteignait le timeout sans avoir assigné de valeur à toutes les variables, ou lorsque la solution trouvée était partielle.

Installation de Gecode : j'ai tenté d'intégrer le solveur Gecode pour disposer d'un troisième solveur de comparaison, mais son installation s'est révélée impossible dans mon environnement, en raison de conflits de dépendances.

Intégration d'OR-Tools : face à l'échec de Gecode, j'ai créé un environnement virtuel dédié pour installer OR-Tools, le solveur de Google. Cette approche a fonctionné et m'a permis de disposer d'un troisième solveur opérationnel pour comparer les performances sur les instances du problème.

Impact limité de l'optimisation : deux objectifs ont été testés : la minimisation de la somme des créneaux et la minimisation du *makespan* (dernier créneau utilisé) . Dans les deux cas, les solutions obtenues n'ont pas montré d'amélioration significative par rapport à la simple recherche d'une solution réalisable. Ce constat s'explique par la nature très contrainte du problème : les distances minimales imposées entre les réunions d'un même agent, combinées à la taille réduite du domaine (12 créneaux pour 20 réunions), laissent très peu de liberté. La première solution réalisable trouvée par le solveur utilise déjà le maximum de créneaux disponibles, ou très proche de celui-ci.

2.4 Expérimentations

Solver	**Solved**	**Unsolved**	**Avg. Time (s)**	**Total Time (s)**
ACE	11	16	0.993	26.820
ACE_OPT	11	16	1.073	28.973
CHOCO	2	25	0.375	10.117
CHOCO_OPT	11	16	0.585	15.788
OR_TOOLS	11	16	0.021	0.668
OR_TOOLS_OPT	11	16	0.042	1.300

OR-Tools est le plus rapide et résout autant d'instances qu'ACE. Choco sans optimisation échoue sur 25 instances, mais avec optimisation il atteint les mêmes résultats que les autres.

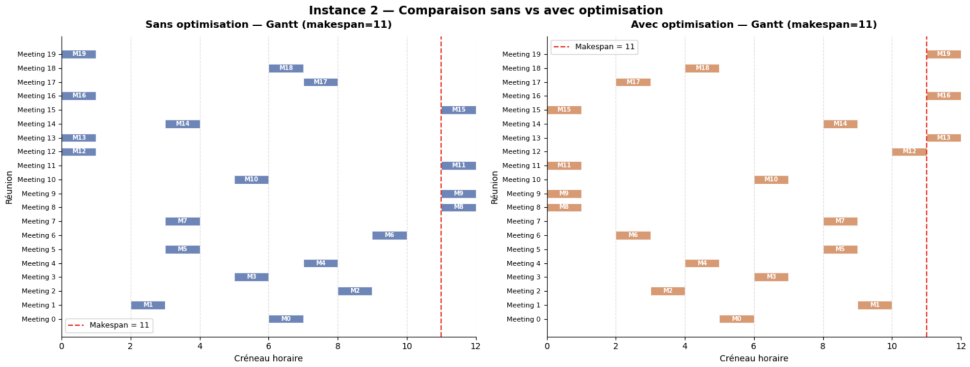

Figure 1: Comparaison ACE / ACE_OPT sur l'instance 2.

Commentaire : Pour l'instance 2, les deux configurations d'ACE atteignent le même makespan (11). En revanche, la distribution des créneaux attribués à chaque réunion diffère : l'optimisation a décalé certaines réunions (par exemple, la réunion 16 passe de 0 à 11 ? en réalité les valeurs changent). Cela illustre que même sans améliorer la valeur du makespan, l'objectif guide le solveur vers une affectation différente des horaires, potentiellement mieux adaptée à d'autres critères (somme des créneaux, équilibrage).